

Coherent and Reversible AI-assisted XR Scene Authoring by Spatial Dependency Graphs

Mine Dastan*
Polytechnic University of Bari

Michele Fiorentino
Polytechnic University of Bari

Dongyu Qiu
Singapore Institute of Technology

Frank Guan
Singapore Institute of Technology

Haiyan Jiang
Hong Kong Polytechnic University

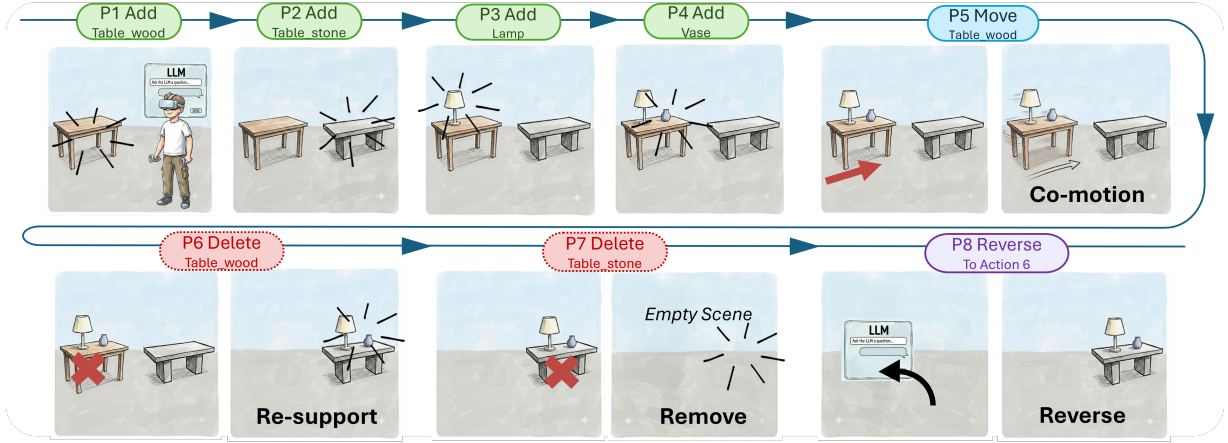


Figure 1: Storyboard of coherent and reversible AI-assisted XR scene authoring: Natural-language prompts (P_n) are recorded in a timeline. **Co-motion** moves dependent objects with their support object, **Re-support** relocates dependent objects to another compatible support when their original support is deleted; and **Removal** removes dependent objects when no compatible support remains. **Reverse** enables previous scene states to be restored through the timeline.

ABSTRACT

The adoption of extended reality technologies as authoring tools is increasing, as XR and AI-assisted systems simplify immersive content creation and make authoring accessible to users without advanced technical expertise or a traditional desktop interface. Existing tools can capture user intent and translate it into immersive outputs. However, their editing capabilities remain limited, as they often lack explicit representations of inter-object dependencies and therefore cannot determine how an action should affect spatially related objects. To preserve scene coherence during authoring actions, we introduce a *Spatial Dependency Graph (SDG)* that represents support dependencies inferred from object bounding-volumes and object-type semantics. The SDG supports three cascaded operation: (i) **co-motion** in case of movements, (ii) **re-support** find a compatible support in scene, and (iii) **remove** dependents when no coherent support exist and a **reverse** which rollback to any prior scene states. We implemented SDG in a VR demonstrator integrating Unity3D and Meta Quest 3 with a text LLM-based command parser. A preliminary feasibility study with two participants produced an adapted SUS score of 82.5/100, although the result should be interpreted only as initial evidence of usability.

Index Terms: Extended Reality, Authoring, Human-AI Interaction, Coherence, Spatial Dependency Graphs.

*e-mail: mine.dastan@poliba.it

1 INTRODUCTION

Authoring tools enable users to create domain-specific content without needing to manage complex implementation details [1]. Modern game engines have made scene authoring more accessible in extended reality (XR) through standardized workflows and reusable components. However, scene authoring remains a challenging and time-consuming task, often requiring extensive manual object manipulation. Recent progress in AI-assisted systems, particularly intelligent agents and large language models (LLMs), has enabled faster, more accessible workflows based on high-level natural-language instructions [2, 8]. Emerging tools such as Unity AI Assistant [11] further show the growing interest in AI-supported authoring.

Despite this progress, current AI-assisted XR authoring systems mainly focus on intent interpretation, action execution, and reviewable AI-generated steps. However, they often lack explicit representations of inter-object dependencies, which limits their ability to reason about how actions should affect spatially related objects. As a result, they may compromise spatial coherence during common authoring workflows such as creating, inspecting, revising, and reversing scene actions that introduce dependent consequences. Explicit dependency reasoning is therefore needed especially in AI-assisted workflows to keep the consequences predictable and inspectable.

This lack of spatial reasoning is particularly evident in support relationships. A support relationship describes a case in which one object depends on another for its intended position, meaning, or stability. For example, a vase may depend on a table as its support object, a book may depend on a shelf, and a lamp may depend on a desk or the floor. When a support object is moved or deleted, the system must determine how its dependent objects should respond: whether they should move with it, be reassigned to another compati-

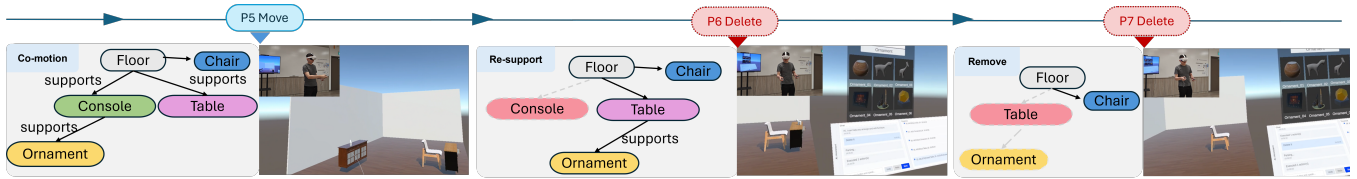


Figure 2: The interface supports interior-scene composition through category-based object selection, natural-language commands, direct manipulation, and a timeline for reviewing and reversing actions. User commands are parsed into structured actions, evaluated through the SDG, and resolved through co-motion, re-support, or removal before being stored in the reversible action timeline.

ble support or be removed through a reversible fallback. Without an explicit policy, AI-assisted actions can leave objects floating, semantically misplaced that break scene coherence, obscure user intent, and require manual correction. In many authoring systems, rollback is treated primarily as state recovery, restoring a previous scene state without reasoning about why an action affected related objects. This further limits support for more meaningful reverse actions, such as undoing only propagated effects or restoring dependent objects according to their original authoring intent.

To address these limitations, this paper introduces a runtime Spatial Dependency Graph (SDG) for coherent and reversible AI-assisted virtual scene authoring. The SDG represents inter-object support as an operational dependency layer that guides how actions propagate through co-motion, re-support, and removal. It is integrated with an action-history mechanism that supports reversing and redoing both direct and cascaded changes. The primary contribution is the SDG-based dependency mechanism, which operates independently of the input modality and supports natural-language commands, menus, and direct manipulation through the same structured action representation. AI converts user input into structured actions, while the SDG predictably resolves their effects on dependent objects. Unlike general scene-graph representations used mainly for grounding, placement, or spatial context, the SDG is designed to preserve spatial coherence during continuous in-HMD authoring [4, 10]. We demonstrate the method in an XR interior-design prototype and report preliminary questionnaire-based feedback from two participants.

2 RELATED WORK

Recent work on AI-assisted XR authoring explores how intelligent agents can support immersive scene creation through natural language, multimodal input, and mixed-initiative interaction. LLMR [2] uses large language models to create and modify mixed-reality scenes through text. VRCopilot [16] integrates generative AI into VR authoring while preserving user agency and iterative control. Other systems support situated manipulation. PITAR [7] combines gaze, pointing, and speech to resolve commands such as “put it there,” while VRMover [12] maps language instructions to object manipulation. EchoLadder [5] presents AI-generated suggestions that users can apply, regenerate, or retract. These works support the expression and refinement of scene actions. Our work instead focuses on what happens after an action is accepted, particularly when other objects depend on the affected object.

Undo, redo, and action history allow users to recover from errors, explore alternatives, and revisit earlier states. Rasch et al. [9] investigated undo for object manipulation in multi-user VR. VR-Git [15] represents history as navigable 3D versions that users can preview, branch, and reuse. VRCopilot [16] and EchoLadder [5] likewise emphasize iteration and reviewable AI outputs. However, these systems mainly represent history as actions, versions, or suggestions. In dependency-aware editing, one action may also move, re-support, or remove dependent objects. We therefore connect undo and redo to the scene’s dependency structure, allowing both direct and propagated effects to be recovered coherently. Causal dependen-

cies have also been used to preserve consistency in asynchronous XR interactions [3]. Our work instead applies support dependencies to coherent and reversible scene authoring.

A related line of work focuses on AI-assisted scene generation. Holodeck [14] and SceneCraft [6] generate complete 3D scenes from text. These systems produce plausible layouts from high-level prompts, but differ from continuous in-HMD editing. XR authoring involves incremental changes made by a user already situated in the scene. The system must therefore preserve coherence as objects are repeatedly created, moved, deleted, revised, undone, and redone. Scene graphs represent spatial and semantic structure. In robotics and embodied AI, they encode objects, labels, relations, and contextual information for perception, planning, and task execution [10, 4]. ImagineAR [8] uses scene understanding to support language-based AR authoring. The proposed SDG follows this graph-based paradigm but has a narrower operational role. It stores support dependencies used to determine how actions propagate and how their effects are reversed. We focus on the *supportedBy* relation because it captures common placement and attachment dependencies. Other relations, such as proximity and functional arrangement, remain future extensions.

Physics-based methods offer another way to maintain spatial plausibility. Physcene [13], for example, uses simulation to generate physically plausible cluttered scenes. Such methods suit offline settings where scenes can settle over time. Live XR authoring instead requires immediate, predictable, and reviewable actions. Our goal is therefore to preserve authoring intent rather than fully simulate physical behavior. For this reason, we use a dependency-aware cascade instead of full physics simulation.

3 METHODOLOGY

The method separates user input from spatial reasoning by converting direct manipulation, interface commands, voice, and text into a common structured action. It comprises a runtime SDG, a dependency-aware cascade mechanism, and a reversible action history. The SDG stores support dependencies, the cascade resolves actions through co-motion, re-support, or removal, and the history records both direct and propagated effects for undo and redo. Each action proceeds through intent formalization, dependency analysis, cascade planning, scene update, and state preservation (Figure 2). We implemented the method as a Meta Quest VR prototype for indoor scene composition.

Intent Formalization Intent formalization converts user input into a structured action command before any spatial reasoning is applied. The system supports both direct interaction and language-mediated input. Direct actions, such as selecting an object from the interface or grabbing an object with the controller, are converted locally into action commands. Spoken or typed natural-language commands are first divided into clauses and then parsed by GPT-4o mini using a structured prompt. The model returns a `ParsedSceneCommand` object containing the operation type, target object, transformation parameters, and optional relocation target. Supported operations include adding, deleting, moving, rotating,

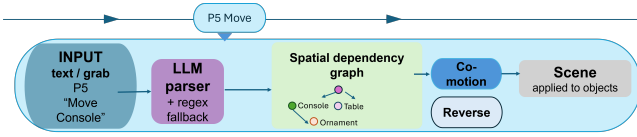


Figure 3: **Coherent and reversible XR scene authoring pipeline.** The SDG processes structured commands and records coherent changes in a reversible timeline.

scaling, selecting, undoing, redoing, and reverting to a previous state. A local rule-based parser provides fallback support for common commands when network access is unavailable. The LLM is deliberately limited to intent parsing. It does not decide how dependent objects should move, where unsupported objects should be reassigned, or whether an object should be removed. These decisions are handled by the SDG and cascade modes described in the following sections. GPT-4o mini is a relatively lightweight language model, although its parameter count has not been publicly disclosed. Because we evaluated only this model, the minimum model scale required for reliable command parsing remains unknown.

Spatial Dependency Graph Construction Scene coherence is represented through the runtime SDG, which stores directed *supportedBy* relations between scene objects. A support object provides the placement context for another object, while a dependent object relies on that support for its intended position, meaning, or stability. The SDG uses these relations to determine how actions propagate.

Each object carries a lightweight `ObjectSemantics` record specifying its type, whether it can act as a support surface, whether it can depend on another object, whether it is movable, and the support categories with which it is compatible. In the current prototype, these attributes are manually assigned by the developers to each Unity prefab when the object is created. This explicit semantic layer prevents the system from relying only on geometry, since a geometrically plausible placement may still be semantically invalid. The language model does not create or modify these rules at runtime.

The SDG is built at runtime from scene geometry rather than from Unity’s transform hierarchy. For every ordered pair of scene objects (A, B) , an edge $A \xrightarrow{\text{supportedBy}} B$ is added when three conditions hold. First, the vertical gap between A ’s bounding-volume bottom face and B ’s top face must lie within a small threshold $\tau_v = 5\text{cm}$. Second, the X/Z-plane footprints must overlap by at least $\tau_o = 1\text{cm}$. Third, B must be a semantically valid support surface for A ’s type, according to a compact compatibility table. When multiple valid support objects exist, each dependent object retains only one support relation: the highest valid surface beneath it. After construction, a topological ordering of the graph provides a deterministic order for applying dependency cascades. The current SDG models only *supportedBy* relations because they provide clear geometric and semantic cues. Relations such as *inFrontOf*, *inside*, and *attachedTo* require different propagation policies and remain future work.

Coherent Scene Authoring Cascade modes define how the scene is updated when an action affects a support object. Instead of leaving dependent objects floating, misplaced, or semantically invalid, the method applies one of three deterministic behaviors: co-motion, re-support, or removal.

Co-motion. When a support object P is translated, rotated, or scaled, its dependent objects are moved with it. Before the transformation, the system stores the relative offset between P and each transitive dependent object D :

$$O_D = M_D M_P^{-1}.$$

After P is updated, each dependent object is repositioned by applying the stored offset to the new pose of P . This keeps dependent

objects aligned with their support object without using Unity re-parenting.

Re-support. When a support object P is deleted, each dependent object D is evaluated for reassignment. The system searches the remaining registered support surfaces for compatible candidates. Candidates are filtered by semantic compatibility and scored using footprint overlap, elevation, horizontal proximity, and height similarity. The floor is included as a candidate support surface and is considered only for objects whose semantic metadata permits floor-level placement. For example, a floor lamp may be compatible with both a desk and the floor, whereas a small table ornament may be restricted to elevated surfaces. Geometrically plausible but semantically incompatible candidates are excluded. The highest-scoring candidate becomes the new support object.

The vertical position of D is recomputed as:

$$y_{\text{new}} = y_{\text{top}}^{\text{support}} + \frac{1}{2}h_D.$$

The X/Z position is preserved when it remains inside the new support footprint; otherwise, the object is centered or clamped to a valid position on the new support.

Removal. If no registered and semantically compatible support surface exists, the dependent object is removed from the active scene. This behavior is a deterministic fallback intended to avoid floating or semantically invalid placements. Its complete state, including its transform, semantic metadata, active state, and previous support relation, remains stored in the action history, so the removal can be reversed later. This differs from physics simulation: instead of letting the object fall, the system treats the absence of a valid support as the loss of the object’s intended placement context.

Reversible Scene Authoring Reversibility is integrated into the authoring pipeline through a per-action timeline. After each action, the system stores the scene state, including each tracked object’s position, rotation, scale, color, semantics, support relation, and active state. The user can step backward, step forward, or return to an earlier state through the in-HMD timeline panel or a natural-language command. Each timeline entry includes both the initiating action and any cascaded effects produced by the SDG. Thus, when an object is moved by co-motion, reassigned by re-support, or removed by the removal cascade, the consequence is stored as part of the same action. This allows reverse and redo actions to restore coherent scene states rather than undoing isolated object transformations. The same mechanism applies to direct manipulation. When the user grabs an object that currently acts as a support object, the system captures it together with its dependent objects as a dependency cluster. During the drag, the system preserves the relative position and orientation of each dependent object, so that dependent objects follow the support object in real time. On release, support relations are re-evaluated, the cascade is finalized, and a new timeline entry is committed.

4 PRELIMINARY FEASIBILITY STUDY

We conducted a preliminary feasibility study to obtain an early indication of whether the proposed dependency-aware authoring workflow is understandable and usable in an interactive XR setting. The objective was not to provide statistically generalizable results, but to verify whether users could complete a representative sequence involving object creation, co-motion, re-support, removal, and reverse actions. Interior design was selected as the application scenario because room composition naturally involves support relations between objects, such as objects placed on surfaces, moved together, reassigned, or restored. This made it suitable for observing whether the SDG preserves spatial coherence during common authoring operations. Two volunteer participants took part in the study, one male and one female, aged 26 and 27. Their mean self-rated VR experience was 4.5/7 (1 = no experience, 7 = extensive experience), and their mean self-reported frequency of LLM tools use

was 6.5/7 (1 = very infrequent use, 7 = very frequent use). After the session, participants completed an adapted System Usability Scale (SUS) questionnaire (7-point scale, from 1 = strongly disagree to 7 = strongly agree). The adapted seven-point SUS responses were normalized to 0–100. No task-completion time, error rate, interaction logs, or observational coding were collected. The study comprised five guided scenarios, each corresponding to a core system function. In **Scene Composition**, participants created a room layout using voice commands, establishing valid support relations in the SDG. In **Co-motion**, they moved a support object and observed its dependent objects move with it. In **Re-support**, they deleted the original support after adding a compatible alternative, prompting dependent objects to be reassigned. In **Removal**, they deleted the remaining support, causing unsupported objects to be removed from the active scene. Finally, in **Reverse**, they used the timeline to restore removed or reassigned objects together with their previous support relations.

5 DISCUSSION

The adapted SUS questionnaire produced a mean score of 82.5/100, suggesting a positive initial perception of the prototype. Participants rated the system favorably for ease of use, confidence, and expected learnability, while negatively worded items related to complexity and cumbersomeness received low scores. Mild reservations appeared for items related to technical support and prior learning. These results should be interpreted only as preliminary feedback. The sample was very small ($n = 2$), the SUS response format was adapted from the validated 5-point scale to a 7-point scale, and no task-completion time, error rate, interaction logs, or observational coding were collected. Therefore, the results cannot support statistically reliable claims about usability or authoring efficiency, but they provide an encouraging indication that the dependency-aware workflow is understandable enough to motivate a larger controlled study.

Limitations and Future Work: The current implementation models only rest-on-top *supportedBy* relations; richer relations such as containment, attachment, and relative positioning may require different propagation policies. Removal is used as a reversible fallback when no compatible support exists, although this may not always match the user’s intent. Future systems could provide previews, user confirmation, or alternative placement strategies. Semantic compatibility is manually assigned, limiting scalability to unseen objects and categories. Future work should investigate automatic semantic inference, compare language-based and direct-manipulation input, and evaluate the SDG against alternative dependency-handling approaches in a larger controlled study.

6 CONCLUSION

We presented an SDG for coherent and reversible XR scene authoring. The graph represents semantically filtered support dependencies and propagates actions through co-motion, re-support, and reversible removal. A per-action timeline stores both direct actions and cascaded effects, enabling reverse and redo actions over coherent scene states. By limiting the LLM to command parsing, the system separates user intent expression from spatial reasoning. Because direct manipulation and language-mediated commands are converted into the same structured action representation, the SDG is independent of the interaction modality and can also be applied in non-LLM-based authoring workflows. Preliminary feedback from two participants suggests that the workflow is understandable, but broader evaluation is required. The work provides a basis for broader dependency-aware XR authoring models involving richer spatial relations, configurable cascade policies and mixed reality dependencies.

SUPPLEMENTARY MATERIALS

A video demonstrating the user experience during preliminary feasibility study is provided as supplementary material.

REFERENCES

- [1] J. Brett and C. Hargood. Authoring tools for mixed reality. In *NHT’23 Workshop at ACM Hypertext 2023*, 2023. 1
- [2] F. De La Torre, C. M. Fang, H. Huang, A. Banburski-Fahey, J. Amores Fernandez, and J. Lanier. Llmr: Real-time prompting of interactive worlds using large language models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pp. 1–22, 2024. 1, 2
- [3] A. R. Fender and C. Holz. Causality-preserving asynchronous reality. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, CHI ’22. Association for Computing Machinery, New York, NY, USA, 2022. doi: 10.1145/3491102.3501836 2
- [4] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa, et al. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5021–5028. IEEE, 2024. 2
- [5] Z. Hou, J. Tian, N. Li, F. Ren, and C. Liu. Echoladder: Progressive ai-assisted design of immersive vr scenes. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–22, 2025. 2
- [6] Z. Hu, A. Iscen, A. Jain, T. Kipf, Y. Yue, D. A. Ross, C. Schmid, and A. Fathi. Scenecraft: An llm agent for synthesizing 3d scenes as blender code. In *Forty-first International Conference on Machine Learning*, 2024. 2
- [7] H. Jiang, D. Qiu, A. Stanesco, Y. Wang, H. B.-L. Duh, and F. Guan. Pitar: an llm-powered agent towards intelligent and accurate manipulations in extended reality with multimodal interactions. In *SIGGRAPH Asia 2025 XR*, pp. 1–2. ACM, 2025. doi: 10.1145/3761667.3761944 2
- [8] J. Lee, F. Aleotti, D. Mazala, G. Garcia-Hernando, S. Vicente, O. J. Johnston, I. Kraus-Liang, J. Powierza, D. Shin, J. E. Froehlich, et al. Imaginator: Ai-assisted in-situ authoring in augmented reality. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–21, 2025. 1, 2
- [9] J. Rasch, F. Perzl, Y. Weiss, and F. Müller. Just undo it: exploring undo mechanics in multi-user virtual reality. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pp. 1–14, 2024. 2
- [10] A. Rosinol, A. Gupta, M. Abate, J. Shi, and L. Carlone. 3d dynamic scene graphs: Actionable spatial perception with places, objects, and humans. *arXiv preprint arXiv:2002.06289*, 2020. 2
- [11] Unity Technologies. Unity AI Assistant Documentation. <https://docs.unity3d.com/Packages/com.unity.ai.assistant@2.12/manual/about/assistant-modes.html>, 2026. 1
- [12] X. E. Wang, Z. P. Sin, Y. Jia, D. Archer, W. H. Fong, Q. Li, and C. Li. Can you move these over there? exploring an llm-based vr mover to support natural multi-object manipulation. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–18, 2025. 2
- [13] Y. Yang, B. Jia, P. Zhi, and S. Huang. Physcene: Physically interactable 3d scene synthesis for embodied ai. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16262–16272, 2024. 2
- [14] Y. Yang, F.-Y. Sun, L. Weihs, E. Vanderbilt, A. Herrasti, W. Han, J. Wu, N. Haber, R. Krishna, L. Liu, et al. Holodeck: Language guided generation of 3d embodied ai environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16227–16237, 2024. 2
- [15] L. Zhang, A. Agrawal, S. Oney, and A. Guo. Vrgit: A version control system for collaborative content creation in virtual reality. In *CHI 2023 - Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, Conference on Human Factors in Computing Systems - Proceedings. Association for Computing Machinery, 2023. doi: 10.1145/3544548.3581136 2
- [16] L. Zhang, J. Pan, J. Gettig, S. Oney, and A. Guo. Vrcopilot: Authoring 3d layouts with generative ai models in vr. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–13, 2024. 2